

DocMind

RAG-Powered Document Q&A; System

Complete Step-by-Step Technical Guide

INGESTION

EMBEDDINGS

RETRIEVAL

GENERATION

EVALUATION

7 comprehensive steps · Full source code · Portfolio-ready

Built with Python · LangChain · ChromaDB · OpenAI · Streamlit

Contents

1	What is RAG? — Architecture Overview
2	Project Structure & File Breakdown
3	Step 1 — Document Loading
4	Step 2 — Text Chunking
5	Step 3 — Embedding — Text to Vectors
6	Step 4 — Vector Database (ChromaDB)
7	Step 5 — Semantic Retrieval
8	Step 6 — Prompt Construction & Generation
9	Step 7 — Streamlit UI & Session Management
10	Evaluation with RAGAS
11	Running & Deployment
12	Upgrade Path to Production
13	Freelance Packaging & Loom Demo Tips

0

1

OVERVIEW

What is RAG?

Retrieval-Augmented Generation (RAG) is a technique that grounds LLM responses in your own documents. Instead of relying solely on the model's training data (which may be outdated or lack domain-specific knowledge), RAG first retrieves relevant passages from a vector database, then feeds them as context to the LLM. The result: accurate, cited, hallucination-resistant answers.

The Two Phases

INGESTION (one-time)

Upload docs → Load pages → Chunk text →
Embed chunks → Store in ChromaDB

QUERY (every question)

User asks → Embed query → Search ChromaDB
→ Build prompt → LLM → Answer + citations

Why RAG beats fine-tuning for most use cases:

- No GPU training required — works entirely with API calls
- Documents can be updated without retraining the model
- Every answer is traceable to a source chunk (auditability)
- Works on private, confidential documents that never touch the LLM's training data
- Cost-effective: embedding a 300-chunk knowledge base costs ~\$0.001

0 PROJECT

2

File Structure & Architecture

```
docmind-rag/
■■■ app.py ← Streamlit UI, session state, chat interface
■■■ rag_pipeline.py ← Core RAG logic (ingest + query)
■■■ utils.py ← Token estimation, text cleaning helpers
■■■ requirements.txt ← All pinned dependencies
■■■ .env.example ← API key template
■■■ chroma_db/ ← Auto-created: persistent vector store
```

Module Responsibilities

File	Responsibility	Key classes/functions
app.py	UI layout, sidebar config, chat display, session state	Streamlit widgets, chat_message, file_uploader
rag_pipeline.py	Document ingestion, embedding, retrieval, generation	RAGPipeline.ingest_files(), .query()
utils.py	Text cleaning, token estimation, source formatting	clean_text(), estimate_tokens()

03

STEP 1

Document Loading

The ingestion pipeline begins by converting raw uploaded files into LangChain Document objects. Each Document has two parts: `page_content` (the extracted text) and `metadata` (filename, page number, file type). We route each file to the correct loader based on its extension.

The `_load_file()` method:

```
def _load_file(self, uploaded_file) -> List[Document]:
    suffix = Path(uploaded_file.name).suffix.lower()
    # Write BytesIO to temp file (LangChain needs a path)
    with tempfile.NamedTemporaryFile(delete=False, suffix=suffix) as tmp:
        tmp.write(uploaded_file.read())
        tmp_path = tmp.name

    if suffix == '.pdf':
        loader = PyMuPDFLoader(tmp_path) # fast, handles scanned PDFs
    elif suffix == '.docx':
        loader = Docx2txtLoader(tmp_path) # strips Word formatting
    elif suffix == '.txt':
        loader = TextLoader(tmp_path) # simple UTF-8 read
    else:
        loader = TextLoader(tmp_path) # simple UTF-8 read

    docs = loader.load()
    for doc in docs:
        doc.metadata['source'] = uploaded_file.name # tag for citations
    return docs
```

Why temp files?

Streamlit uploads are in-memory BytesIO objects. LangChain loaders expect file system paths. We write to a temp file, load it, then delete it immediately.

Metadata is critical

The source filename stored in metadata is what powers 'Cited from: contract.pdf' in the final answer. Never lose this during processing.

04

STEP 2

Text Chunking

LLMs have context window limits — you cannot pass a 50-page PDF as input. The solution is to split documents into smaller overlapping chunks. Overlap is crucial: it prevents an important sentence from being severed between two chunks, ensuring completeness of retrieved content.

RecursiveCharacterTextSplitter configuration:

```
splitter = RecursiveCharacterTextSplitter(  
    chunk_size=512, # ~380 words per chunk  
    chunk_overlap=64, # ~48 shared words with next chunk  
    separators=[  
        "\n\n", # try paragraph breaks first  
        "\n", # then single line breaks  
        ". ", # then sentence ends  
        " ", "" # fallback: word level, then char level  
    ]  
)
```

Chunking Strategy Tradeoffs

Chunk Size	Retrieval Precision	Context Richness	Best For
256 tokens	High	Low	Q&A on short docs, FAQs
512 tokens	Balanced ✓	Balanced ✓	Most use cases (default)
1024 tokens	Low	High	Summarisation tasks

Why 'Recursive'?

Unlike simple character splitting, RecursiveCharacterTextSplitter tries large separators (paragraphs) first. Only if a paragraph is still too large does it try sentences, then words. This preserves semantic coherence — a paragraph about 'termination conditions' stays together.

05

STEP 3

Embedding — Text to Vectors

Every text chunk is converted into a dense numerical vector (an array of 1536 floats) that encodes its semantic meaning. Chunks with similar meanings get similar vectors. This is the mathematical foundation of semantic search — 'termination clause' and 'how to exit the contract' will have nearby vectors even though they share no exact words.

Embedding setup in rag_pipeline.py:

```
self.embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small",
    openai_api_key=os.environ["OPENAI_API_KEY"]
)

# Embed all chunks and store in ChromaDB (one API call batch)
vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=self.embeddings,
    persist_directory="./chroma_db"
)
```

Model	Dimensions	Cost/1K tokens	Notes
text-embedding-3-small	1536	\$0.00002	Default — best value ✓
text-embedding-3-large	3072	\$0.00013	Higher quality, 6x cost
all-MiniLM-L6-v2	384	Free (local)	HuggingFace, offline use

06

STEP 4

Vector Database — ChromaDB

ChromaDB is a local, file-based vector database. It stores each chunk's vector alongside its original text and metadata. When a query vector arrives, ChromaDB uses cosine similarity to find the nearest neighbours in milliseconds — even across hundreds of thousands of chunks.

Storing and querying:

```
# After ingestion, ChromaDB persists to disk automatically
# chroma_db/ directory contains the SQLite + vector index

# Querying returns (Document, score) tuples
results = vectorstore.similarity_search_with_relevance_scores(
    query="What are the termination clauses?",
    k=4 # retrieve top 4 chunks
)

for doc, score in results:
    print(f'Score: {score:.2f} | {doc.metadata["source"]}')
# Score: 0.91 | acme_contract.pdf
# Score: 0.85 | supplier_agreement.pdf
```

Choosing your vector database:

Database	Type	When to use
ChromaDB	Local / embedded	Portfolio demos, dev, small clients
FAISS	Local / in-memory	High-speed local search, no persistence needed
Pinecone	Cloud / managed	Production SaaS, multi-tenant, scale
Weaviate	Self-hosted / cloud	Enterprise, GraphQL API, hybrid search

07

STEP 5

Semantic Retrieval

When a user submits a query, it is embedded using the same model used during ingestion. This creates a query vector that lives in the same semantic space as the stored chunk vectors. ChromaDB then returns the top-K chunks by cosine similarity score.

Full retrieval flow in query():

```
def query(self, question, chat_history=None):
    # 1. Retrieve top-k chunks with scores
    results = self.vectorstore.similarity_search_with_relevance_scores(
        question, k=self.top_k
    )

    # 2. Build context string and source metadata
    context_parts, sources = [], []
    for i, (doc, score) in enumerate(results):
        src = doc.metadata.get('source', 'unknown')
        chunk_id = doc.metadata.get('chunk_id', i+1)
        context_parts.append(
            f'[Source: {src}, Chunk {chunk_id}]\n{doc.page_content}'
        )
        sources.append({
            'source': src, 'chunk_id': chunk_id,
            'score': round(score, 3),
            'preview': doc.page_content[:200]
        })
    context = '\n\n---\n\n'.join(context_parts)
```

Understanding Similarity Scores

Scores range from 0.0 (completely unrelated) to 1.0 (identical). A score above 0.75 is a strong match. Scores below 0.5 usually indicate the document doesn't contain information relevant to the query — a good signal to tell the user 'this topic isn't covered in the uploaded documents'.

08

STEP 6

Prompt Construction & Generation

The retrieved chunks are packed into a carefully structured prompt alongside the user's question and recent chat history. The LLM is given strict instructions to answer only from the provided context. This is the core anti-hallucination mechanism in RAG.

System prompt (in rag_pipeline.py):

```
SYSTEM_PROMPT = """You are DocMind, an expert document analyst.
Answer questions strictly based on the provided context chunks.
Rules:
1. Only use information present in the context. Do not hallucinate.
2. If the answer is not in the context, say so explicitly.
3. Always cite the source document name.
4. Be concise and precise. Use bullet points for lists.
5. For legal/contract documents, note any caveats."""
```

RAG prompt template:

```
RAG_PROMPT_TEMPLATE = '''Context from documents:
{context} # ← the retrieved chunks

Chat History:
{chat_history} # ← last 3 turns for memory

User Question: {question}
Answer:'''

# Build the messages and call the LLM
messages = [
    SystemMessage(content=SYSTEM_PROMPT),
    HumanMessage(content=prompt),
]

response = self.llm.invoke(messages) # temperature=0.1
```

Why temperature=0.1?

Low temperature makes the model follow instructions literally and stick to the context rather than improvising. Essential for legal or financial documents.

Conversation memory

The last 3 turns of chat history are included so the model understands follow-up questions like 'what about clause 5?' without re-uploading context.

09

STEP 7

Streamlit UI & Session Management

app.py handles all UI concerns: file upload, configuration sliders, chat rendering, source card display, and session state. Streamlit re-runs the entire script on every interaction, so all persistent data lives in `st.session_state`.

Session state architecture:

```
# Initialised once at script top
if 'rag' not in st.session_state:
    st.session_state.rag = None # RAGPipeline instance
if 'chat_history' not in st.session_state:
    st.session_state.chat_history = [] # List of {role, content, sources}
if 'docs_loaded' not in st.session_state:
    st.session_state.docs_loaded = False # Gate for the chat UI

# Build knowledge base when button clicked
if st.button('Build Knowledge Base'):
    rag = RAGPipeline(chunk_size, chunk_overlap, top_k, model)
    stats = rag.ingest_files(uploaded_files)
    st.session_state.rag = rag
    st.session_state.docs_loaded = True
```

Chat loop with source display:

```
if prompt := st.chat_input('Ask anything about your documents...'):
    # 1. Add user message to history
    st.session_state.chat_history.append({'role': 'user', 'content': prompt})

    # 2. Query the RAG pipeline
    result = st.session_state.rag.query(
        prompt, chat_history=st.session_state.chat_history[:-1]
    )

    # 3. Display answer + source cards
    st.markdown(result['answer'])
    with st.expander(f'Sources ({len(result["sources"])} chunks)'):
        for src in result['sources']:
            st.markdown(f'Score: {src["score"]} | {src["source"]}')

```

10

EVALUATION

Quality Testing with RAGAS

Before delivering to a client, you need to quantify quality. RAGAS (Retrieval Augmented Generation Assessment) is the standard evaluation framework. It measures four dimensions that cover the full pipeline — not just the final answer quality.

The four RAGAS metrics:

Metric	What it measures	Target	If low, fix by...
Faithfulness	Answer is supported by retrieved context (no hallucination)	> 0.85	Tighten system prompt, lower temp
Answer Relevancy	Answer addresses the actual question	> 0.80	Better prompt, more context
Context Precision	Retrieved chunks are actually relevant	> 0.75	Reduce chunk size, tune top-k
Context Recall	All relevant chunks were retrieved	> 0.70	Increase top-k or chunk overlap

1

1

DEPLOY

Running & Deployment

Local setup — 3 commands:

```
# 1. Install all dependencies
pip install -r requirements.txt

# 2. Set your API key
cp .env.example .env
# Edit .env → OPENAI_API_KEY=sk-your-key-here

# 3. Launch the app
streamlit run app.py

# Opens at http://localhost:8501
```

Free cloud deployment on Streamlit Community Cloud:

```
# 1. Push to a public GitHub repo
git init && git add . && git commit -m 'initial'
git push origin main

# 2. Go to share.streamlit.io
# → New app → select your repo → app.py

# 3. Add secrets in the Streamlit dashboard:
# Settings → Secrets → paste OPENAI_API_KEY=sk-...

# → Live public URL in ~2 minutes. Free forever.
```

1

2

ADVANCED

Production Upgrade Path

Each upgrade is a targeted code change — the core RAG logic stays identical. This is how you grow the same project from a portfolio demo to a billable SaaS product.

↑ **ChromaDB → Pinecone**

Swap the vectorstore instantiation. Same `.from_documents()` API. Adds cloud hosting, multi-user isolation, and billion-vector scale.

↑ **Add Hybrid Search (BM25 + Semantic)**

Use LangChain's `EnsembleRetriever` combining a `BM25Retriever` with the Chroma retriever. Dramatically improves recall on exact terms (contract numbers, SKUs, names).

↑ **Add Cross-Encoder Re-ranking**

After retrieving top-20 chunks semantically, run a cross-encoder (`ms-marco-MiniLM-L-6-v2`) to re-rank them. Only the top-4 go to the LLM. Precision improvement with no extra API cost.

↑ **FastAPI + React frontend**

Move the RAGPipeline to a FastAPI backend with `/ingest` and `/query` endpoints. Build a React chat UI. Enables multi-user auth, rate limiting, and white-label delivery.

↑ **Add RAGAS CI evaluation**

Run RAGAS on a golden test set (20 Q&A; pairs) in a GitHub Actions workflow. Block deploys if faithfulness drops below 0.80. Clients love this.

1 BUSINESS

3

Freelance Packaging & Loom Demo Tips

The code is only half the product. The other half is how you present it to clients. A 2-minute Loom video + a deployed demo link is your highest-leverage freelance asset.

Loom demo script (2 minutes):

0:00 – 0:20	Hook: Open with: 'Imagine reviewing 50 contracts without reading a single one.' Show the empty app.
0:20 – 0:50	Upload: Drag in 2–3 contract PDFs. Show the 'Building Knowledge Base' spinner and chunk stats.
0:50 – 1:20	Query: Ask a specific question: 'What is the payment term in the Acme contract?' Show the answer and source card.
1:20 – 1:45	Multi-doc: Ask a cross-document question: 'Which contract has the strictest IP clause?' Show multiple sources.
1:45 – 2:00	CTA: End with: 'I can build this for your team in 2–3 weeks. DM me.'

Target client verticals:

Industry	Documents	Example query	Price signal
Law firms	Contracts, NDAs, briefs	'What are the termination clauses?'	\$2–5K project
Startups	SOPs, handbooks, wikis	'What is the onboarding process?'	\$1–3K project
E-commerce	Product manuals, specs	'What warranty does X carry?'	\$1–2K project
Finance / PE	Reports, filings, memos	'Summarise the risk factors'	\$3–8K project

Built with LangChain · ChromaDB · OpenAI · Streamlit · Python
DocMind — RAG Q&A; System Guide